

CLAUDE.md — Helios

Operating manual for Claude Code (and any AI agent) working in this repo. Read this first, every session. It is the **continuity anchor**: canonical vocabulary, the non-negotiable rules, conventions, and where to find depth. Keep it updated as artifacts land.

1. What Helios is

Helios is the Autonomous Growth Diagnosis Engine — an always-on "AI Growth Analyst" that diagnoses *why* an e-commerce funnel moved, distinguishes **mix-shift from rate-change**, prices the movement in **dollars of revenue-at-risk**, prescribes a **prioritized, statistically-defensible experiment backlog**, and ships an executive **Decision Brief** — all grounded in **governed SQL it never authors by hand** and graded by an **offline eval benchmark**.

It runs on the public GA4 dataset `bigquery-public-data.ga4_obfuscated_sample_ecommerce` (Google Merchandise Store, ~2020-11-01 → 2021-01-31).

Anti-product stance (do not drift into these): Helios is **not** a BI dashboard, **not** an ad-hoc SQL chatbot, **not** a generic "ask-your-data" tool. Conversation is a *secondary* drill-down surface. The product's heartbeat is the **autonomous scheduled run**.

2. Source of truth & precedence

Doc	Role
<code>docs/architecture/HELIOS_PROJECT_BIBLE.md</code>	The full 25-section spec (what every artifact is). Rebuild-from-scratch reference.
<code>docs/planning/DEPENDENCY_MAP.md</code>	Build order, dependencies, keystones, what-to-build-next (T0–T13, milestones M0–M12).
This file	Day-to-day operating rules + canonical vocabulary.

Precedence: the Bible's **Canonical Reference Card** wins over any prose anywhere. This file mirrors it; if they ever disagree, the Reference Card is correct and both should be fixed.

3. The five non-negotiable principles

- Grounding over generation.** The LLM **never authors raw SQL** and **never computes a statistic** in prose. It composes governed metrics via `semantic-mcp` and calls deterministic tools via `stats-mcp`.
- Verify-then-trust.** Every query is `dry_run` -cost-checked + schema-validated; every result is `reconcile` -checked against canonical totals; every finding is attacked by the **Critic** before it ships.
- Determinism where it matters.** All math (decomposition, significance, power, forecasting) runs in real Python (`scipy/statsmodels/prophet`), seeded — never in token-space.
- Every finding is actionable.** A finding without a significance test, a dollar impact, and a recommended action is not a finding.
- Proactive, not reactive.** Build for the scheduled autonomous run first; conversation is secondary.

Grounding rules G1–G5 (enforced structurally + behaviorally)

- G1** — Never emit raw SQL. To get data, call `semantic-mcp.build_query(metric, dims, filters, window)` or `get_metric(name)`.
- G2** — Never compute a statistic in prose. Anomaly scores, decompositions, significance, forecasts, power → `stats-mcp` / `experiment-mcp` only. Numbers in the brief are tool outputs verbatim.
- G3** — `warehouse-mcp.dry_run` **before** every `run_query`. Over-budget ⇒ narrow window/dims, do not retry blindly.
- G4** — Reconcile aggregates against `warehouse-mcp.reconcile`; >0.5% drift fails the finding.
- G5** — Use **only** canonical metric/dimension names. An unknown name is a hard error, not a fallback to free SQL.

4. Canonical vocabulary (NEVER paraphrase — these are physical names)

Macro funnel (session-scoped; session key = (user_pseudo_id, ga_session_id))

```
session_start → view_item → add_to_cart → begin_checkout → add_shipping_info → add_payment_info → purchase
```

Report step-to-step rates **and** overall `session_conversion_rate = purchasing_sessions / sessions` .

Metrics (snake_case)

`sessions` , `users` , `new_users` , `returning_users` , `engaged_sessions` , `engagement_rate` , `view_item_sessions` , `add_to_cart_sessions` , `begin_checkout_sessions` , `purchasing_sessions` , `session_conversion_rate` , `view_to_cart_rate` , `cart_to_checkout_rate` , `checkout_to_purchase_rate` , `cart_abandonment_rate` , `checkout_abandonment_rate` , `transactions` , `revenue` , `gross_revenue` , `net_revenue` , `aov` , `items_per_transaction` , `revenue_per_session` (RPS), `revenue_per_user` (ARPU).

Dimensions

`device_category` , `operating_system` , `browser` , `country` , `region` , `channel_group` , `source` , `medium` , `campaign` , `landing_page` , `item_category` , `item_name` , `is_new_user` , `day` , `week` , `session_number_bucket` .

Channel groups — exactly 10 (no "Paid Other")

`Direct` , `Organic Search` , `Paid Search` , `Display` , `Paid Social` , `Organic Social` , `Email` , `Affiliates` , `Referral` , `Other` .

MCP servers & tools

Server	Role	Tools
warehouse-mcp	Sole BigQuery client (HTTP)	<code>list_tables</code> , <code>describe_table</code> , <code>dry_run</code> , <code>run_query</code> , <code>reconcile</code>
semantic-mcp	Only path to SQL (stdio)	<code>get_metric</code> , <code>list_dimensions</code> , <code>build_query</code>
stats-mcp	Only path to math (stdio)	<code>detect_anomaly</code> , <code>decompose_change</code> , <code>significance_test</code> , <code>forecast</code> , <code>cohort_retention</code> , <code>rfm_segment</code>
experiment-mcp	Powered backlog (stdio)	<code>power_analysis</code> , <code>runtime_estimate</code> , <code>design_experiment</code>
report-mcp	Brief + memory (stdio)	<code>render_brief</code> , <code>export</code> , <code>save_diagnosis</code> , <code>recall_prior</code>

Agents (7) — Claude Agent SDK, plan-execute-critique

`Orchestrator` (Opus), `Monitor` (Sonnet), `Decompose` (Sonnet), `Diagnose` (Opus), `Prescribe` (Sonnet), `Narrator` (Sonnet), `Critic` (Opus). Per-agent MCP tool allow-lists are enforced (Bible \$18.9) — e.g. the Narrator cannot call `run_query` .

Core decomposition identity (the technical centerpiece — `stats-mcp.decompose_change`)

For aggregate rate $R = \sum_i (w_i \cdot r_i)$ (segment weight × segment rate), ΔR from $t_0 \rightarrow t_1$ splits as:

```
mix_effect    =  $\sum \Delta w_i \cdot r_i(t_0)$       # traffic composition changed
rate_effect   =  $\sum w_i(t_0) \cdot \Delta r_i$      # in-segment behavior changed
interaction   =  $\sum \Delta w_i \cdot \Delta r_i$      # both moved together
 $\Delta R = \text{mix\_effect} + \text{rate\_effect} + \text{interaction}$ 
```

This is how Simpson's paradox is dissolved. Drill into **rate** effects (real behavior change), not **mix** effects (composition artifacts).

Success-metric targets

Root-cause accuracy **≥85%** (vs **≤45%** naive baseline); time-to-diagnosis **<5 min/run**; **0** hallucinated columns/metrics (100% governed SQL); **100%** of findings carry significance + dollar impact; query cost per run **≤ 5 GiB** (the fixed byte budget).

5. dbt & data conventions

- **Layers / prefixes:** `stg_<source>__<entity>` (staging) → `int_<source>__<entity>` (intermediate) → `fct_*` / `dim_*` (marts) → `models/semantic`. Source group: `src_ga4`. snake_case everywhere.
- **Materializations:** staging = `view`; intermediate = `ephemeral`; marts/core = `incremental` (`insert_overwrite`, partition by `event_date`, cluster by `device_category`, `channel_group`); finance/growth = `table`; semantic = `view`.
- **Funnel flags are `reached_*`, max-downstream (monotonic).** `reached_add_to_cart` = the session reached add_to_cart **or any later stage**. This guarantees `sessions ≥ reached_view_item ≥ ... ≥ reached_purchase`, so step rates are always ≤ 1 . The names `did_*` are **retired**.
- **Session key (one canonical expression):** `session_key = TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST(ga_session_id AS STRING))))`; `sessions = COUNT(DISTINCT session_key)`. Never `FARM_FINGERPRINT`, never `COUNT(*)`.
- **Engaged session:** `session_engaged = '1' OR engagement_time_msec ≥ 10000` (use $\geq$, no extra clauses).
- **traffic_source gotcha:** event-level `traffic_source.*` is **user first-touch**, not session source. Prefer session-scoped `event_params.source/medium`; fall back to `traffic_source` only when null.
- **Money & rates:** use only GA4 `_in_usd` columns; never aggregate non-USD twins. Compute rates as `SUM(numerator)/SUM(denominator)` after grouping — **never** an average of per-segment ratios (this is the Simpson's-paradox defense).
- **Single sources of truth:** channel grouping lives **only** in the `channel_group_case()` macro (`macros/channel_group.sql`); metric/dimension definitions live **only** in `models/semantic/semantic_layer.yaml` (the v2 registry; the retired v1 `semantic_models.yaml` is archived in `docs/archive/superseded/`). `semantic-mcp`'s registry: in `mcp_servers.yaml` must point at that exact path.

6. Repository layout

```
helios/
├─ README.md · CLAUDE.md (this file) · .gitignore · MIGRATION_REPORT.md
├─ docs/
│   └─ architecture/ HELIOS_PROJECT_BIBLE · DATA_MODEL · DBT_GUIDE · MCP_ARCHITECTURE · AGENT_ARCHITECTURE ·
METRIC_DEPENDENCY_GRAPH · METRIC_GOVERNANCE_GUIDE
│   └─ planning/ DEPENDENCY_MAP · DEVELOPMENT_PLAN · IMPLEMENTATION_PLAYBOOK · LEAN_SCOPE
│   └─ strategy/ CLAUDE_CODE_WORKFLOW · INTERVIEW_GUIDE · RED_TEAM_REVIEW
│   └─ archive/ (frozen intermediate fragments + superseded/semantic_models.yaml)
├─ models/semantic/ semantic_layer.yaml # the governed registry (v2, 47 metrics)
├─ eval/
│   └─ scenarios/ scenarios.yaml + 01-07_*.yaml + _VALIDATION.md (50-scenario benchmark)
│   └─ benchmark_results/
├─ dbt/ · mcp/ · agents/ · backend/ · frontend/ · tests/ · scripts/ · notebooks/ # implementation code -
empty until M0+ (each has a placeholder README)
```

BigQuery datasets

`bigquery-public-data.ga4_obfuscated_sample_ecommerce` (read-only source) · `helios` marts (`helios.marts.*`) · `helios_memory` · `helios_eval` · dbt targets `helios_dev` / `helios_prod`.

7. Commands

Platform is **Windows / PowerShell**. The repo is currently **documentation + assets only** (no implementation code yet). The commands below are the *canonical interface* — wire them up as each milestone lands (see `docs/planning/DEPENDENCY_MAP.md §3`). Mark a command "live" in this file once it actually works.

```
# dbt (once M0-M5 exist)
dbt deps
dbt build                # full DAG + tests
dbt build --select staging # one layer
dbt test --select fct_funnel # one model's tests
dbt build --select +fct_daily_funnel # model + upstream

# MCP servers (once M6 exists) - stdio servers
python -m helios.mcp.semantic
python -m helios.mcp.stats
python -m helios.mcp.experiment
python -m helios.mcp.report
python -m helios.mcp.warehouse # streamable-http

# Eval harness (once M10 exists)
python -m helios.eval.runner --smoke # 12-scenario subset (every push)
python -m helios.eval.runner --full  # all 50 scenarios (PR to main)
```

Required environment variables

`GOOGLE_APPLICATION_CREDENTIALS` (BigQuery SA, read-only) · `HELIOS_WH_TOKEN` (warehouse-mcp bearer) · `ANTHROPIC_API_KEY` (agents).

8. Keystones — get these right, test them hard (they fail SILENTLY)

1. `int_ga4_sessionized` + `int_ga4_funnel_steps` — if sessionization or the `reached_*` flags are wrong, every downstream number is silently wrong.
2. `models/semantic/semantic_layer.yaml` — the governance keystone; all governed SQL derives from it.
3. `stats-mcp.decompose_change` — unit-test against hand-worked mix/rate/interaction examples (golden values).
4. **Revenue reconciliation** — `fct_orders` revenue must match the source to the cent (`revenue_reconciles` test).
5. `fct_daily_funnel` aggregates `fct_funnel` (which carries `session_revenue`) — not `int_ga4_funnel_steps`. Dropping revenue here breaks the eval's dollar-at-risk labels.

The **CI eval gate** (`.github/workflows/ci.yml` + `eval/gates.yaml`) is the regression firewall: once green, no change may drop top-1 accuracy >2pts or introduce any hallucination.

9. How to use Claude Code on this repo

- **Plan first.** For any multi-file change, use plan mode and check it against `DEPENDENCY_MAP.md` build order before editing. Never build an artifact before its dependencies (the map is a valid topological sort).
- **TDD on SQL.** Write the dbt test / golden-value test *first*, then make it pass. Keystone transforms get golden tests because they fail silently.
- **Eval-driven dev for agents.** Changes to `models/`, `semantic/`, `agents/`, or `eval/` must keep the benchmark $\geq 85\%$ top-1 and hallucination = 0. Run the smoke subset locally before pushing.
- **Parallelize independent tracks** (subagents): the data spine, the data-independent math (`stats-mcp` / `experiment-mcp`), and memory/finance can progress concurrently (`DEPENDENCY_MAP` §7).
- **Keep docs in lockstep.** When you add/rename an artifact, update `DEPENDENCY_MAP.md` and, if a canonical name changes, the Bible's Reference Card and this file. These three are the resume points for the next session.

Do / Don't

- ☒ Compose metrics via `semantic-mcp`; route all math through `stats-mcp`; `dry_run` before `run_query`.
- ☒ Use exact canonical names; add new metrics by editing the registry YAML, not by hand-writing SQL.
- ☒ Make every finding carry significance + dollar impact + an action.
- ☒ Don't hand-author SQL in agent prompts or Python, invent metric synonyms, compute stats in prose, or skip the Critic.
- ☒ Don't add an 11th channel group, reintroduce `did_*` flags, or use a non-canonical session-key expression.
- ☒ Don't widen scope into a dashboard/chatbot — that's the anti-product.

10. Current status & next step

Built: the full documentation suite — `docs/architecture/` (Bible, DATA_MODEL, DBT_GUIDE, MCP/AGENT architecture, metric governance + dependency graph), `docs/planning/` (DEPENDENCY_MAP, DEVELOPMENT_PLAN, IMPLEMENTATION_PLAYBOOK, LEAN_SCOPE), `docs/strategy/` (CLAUDE_CODE_WORKFLOW, INTERVIEW_GUIDE, RED_TEAM_REVIEW) — plus `models/semantic/semantic_layer.yaml` (47-metric registry) and `eval/scenarios/` (50-scenario benchmark). Repo restructured 2026-06-03 (see `MIGRATION_REPORT.md`). **No implementation code yet.**

Next (milestone M0 → M1): repo scaffold + GCP/IAM/ADC + `dbt_project.yaml` / `profiles.yaml` / `packages.yaml`, then the four macros (`channel_group_case` first), `src_ga4.yaml`, and the seed. Front-load the keystones (sessionization, semantic registry) and start `stats-mcp.decompose_change` in parallel.